

A interface *via CLI*

Como usar o Codex no terminal: instalar e fazer login, usar a interface interativa, controlar sandbox e aprovações, automatizar com `codex exec` e configurar tudo no `config.toml`.

`codex`

`Slash commands`

`Sandbox e aprovações`

`codex exec`

`/import do Claude Code`

O Codex *no terminal*

A Codex CLI é a forma mais direta e mais configurável de usar o agente. Ela roda no seu repositório local e usa as ferramentas já instaladas na máquina. Para quem vem do Claude Code, é a superfície mais parecida.

REPOSITÓRIO LOCAL

Trabalha nos seus arquivos

Inspeciona arquivos, faz edições e roda as ferramentas do projeto: testes, lint, build.

CONTROLE

Você escolhe tudo

Modelo, esforço de raciocínio, permissões e comandos adequados a cada tarefa, por flag ou no meio da sessão.

AUTOMAÇÃO

Scripts e CI

Use de forma interativa ou chame `codex exec` em workflows e pipelines repetíveis.

Use a CLI quando...

- **Você trabalha no terminal:** explorar, editar e rodar o repositório num único ciclo.
- **Você precisa de script ou CI:** rodar tarefas não interativas de forma repetível.
- **Você quer code review local** antes de commitar ou abrir um PR.
- **Você quer mandar trabalho para a nuvem** e voltar ao terminal depois (`codex ccloud`).

Instalando e fazendo *login*

Escolha um dos métodos de instalação. Para atualizar, rode o mesmo comando de novo (ou `brew upgrade`).

MACOS / LINUX: INSTALADOR OFICIAL

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

WINDOWS (POWERSHELL)

```
powershell -ExecutionPolicy Bypass -c "irm  
https://chatgpt.com/codex/install.ps1 | iex"
```

NPM • HOMEBREW

```
npm install -g @openai/codex  
brew install --cask codex
```

LOGIN E PRIMEIRO USO

```
cd meu-projeto  
codex                                # 1ª vez: "Sign in with ChatGPT"
```

alternativas de autenticação

```
codex login --device-auth           # sem navegador (SSH)  
printenv OPENAI_API_KEY | codex login --with-api-key  
codex login status                  # mostra o modo ativo
```

Primeiro prompt sugerido: `Tell me about this project`. E crie **checkpoints no Git** antes e depois de cada tarefa.

Com chave de API você usa a CLI normalmente, mas **sem os recursos de nuvem** (aula 1). No Linux, o sandbox precisa do `bubblewrap`.

Dominando a *interface interativa*

Ao rodar `codex`, você entra numa interface de terminal (TUI) com um composer. Estes atalhos mudam o ritmo de trabalho:

ATALHO	O QUE FAZ
<code>@</code>	Busca um arquivo do workspace e insere o caminho no prompt
<code>!</code> no início da linha	Roda um comando de shell local, respeitando o sandbox e as aprovações
<code>Enter</code> durante a execução	Injeta instruções novas na rodada atual
<code>Tab</code> durante a execução	Enfileira um prompt, slash command ou comando para a próxima rodada
<code>Esc</code> <code>Esc</code>	Com o composer vazio: edita a mensagem anterior e bifurca o chat dali
<code>Ctrl</code> + <code>R</code>	Busca no histórico de prompts
<code>Ctrl</code> + <code>O</code>	Copia a última resposta concluída (mesmo que <code>/copy</code>)
<code>Ctrl</code> + <code>C</code>	Encerra a sessão (mesmo que <code>/exit</code>)

EXEMPLO DE SESSÃO

```
> @src/utils/date.py falha com datas
"28/09/2026". Corrija e adicione
um teste.

> !pytest -q tests/test_date.py
# roda o teste sem sair do Codex

> /diff
# confere as edições antes do commit
```

Setas `↑` e `↓` recuperam rascunhos anteriores do composer.

Sandbox e *aprovações*

Dois controles independentes. O **sandbox** (`-s`) limita o que os comandos gerados pelo modelo podem fazer. A **política de aprovação** (`-a`) define quando o Codex para e pergunta antes de executar.

```
--SANDBOX READ-ONLY
```

Só leitura

Inspeciona o projeto sem editar nada. Bom para explorar código desconhecido.

```
--SANDBOX WORKSPACE-WRITE
```

Escreve no workspace

Edita dentro do projeto (e das pastas liberadas com `--add-dir`) e das pastas temporárias. É o equilíbrio para o dia a dia.

```
--SANDBOX DANGER-FULL-ACCESS
```

Sem sandbox

Remove as restrições locais. Use só quando o acesso amplo for intencional.

```
# pede aprovação quando precisar sair do limite
codex -s workspace-write -a on-request
```

```
# nunca pergunta (use em ambiente controlado)
codex -s read-only -a never
```

```
# no meio da sessão
/permissions
```

`--yolo` (`--dangerously-bypass-approvals-and-sandbox`) roda **todo comando sem aprovação e sem sandbox**. A documentação recomenda usar só dentro de um ambiente já isolado, como um container ou runner descartável.

Se o revisor automático recusar uma ação, `/approve` libera uma nova tentativa, uma única vez.

Slash commands *essenciais*

Digite `/` para abrir a lista. Com uma rodada em andamento, `Tab` enfileira o comando para a próxima.

COMANDO	PARA QUÊ	COMANDO	PARA QUÊ
<code>/init</code>	Gera um <code>AGENTS.md</code> inicial no diretório atual	<code>/status</code>	Configuração da sessão e uso de tokens
<code>/model</code>	Troca o modelo (e o esforço de raciocínio)	<code>/compact</code>	Resume o chat para liberar contexto
<code>/permissions</code>	Ajusta o que o Codex faz sem perguntar	<code>/new</code> · <code>/clear</code>	Novo chat · limpa o terminal e recomeça
<code>/plan</code> · <code>/goal</code>	Modo de planejamento · objetivo persistente	<code>/resume</code> · <code>/fork</code>	Retoma um chat salvo · bifurca o atual
<code>/review</code> · <code>/diff</code>	Review da working tree · diff do Git	<code>/mcp</code> · <code>/skills</code>	Ferramentas MCP · skills disponíveis
<code>/mention</code>	Anexa um arquivo ao chat	<code>/app</code>	Continua a sessão no app desktop (aula 2)

Vindo do Claude Code? O comando `/import` abre um seletor para migrar do **Claude Code** (ou do Cursor) a configuração, os arquivos de projeto e os chats recentes (até 50 chats dos últimos 30 dias). Rode numa sessão local, sem nenhuma tarefa em andamento.

Flags úteis ao *iniciar*

As flags globais valem para o comando `codex` e, em geral, também para os subcomandos. Um `PROMPT` opcional já inicia a sessão com a mensagem.

FLAG	EFEITO
<code>-m</code> , <code>--model</code>	Troca o modelo configurado
<code>-i</code> , <code>--image</code>	Anexa imagens ao primeiro prompt (print de erro, diagrama)
<code>-C</code> , <code>--cd</code>	Define o diretório de trabalho do agente
<code>--search</code>	Liga a busca na web ao vivo (o padrão é a busca em cache)
<code>--add-dir</code>	Dá permissão de escrita a outra pasta
<code>-p</code> , <code>--profile</code>	Aplica um perfil de configuração salvo
<code>-c</code> , <code>--config</code>	Sobrescreve uma chave do config.toml só nesta execução
<code>--oss</code>	Usa um modelo aberto local via Ollama ou LM Studio

```
# prompt inicial + imagem do erro
codex -i erro.png "Explique este stack trace"

# outro diretório, com busca na web
codex -C ~/projetos/api --search

# override pontual de configuração
codex -c model_reasoning_effort="high"

# modelo local (sem nuvem)
codex --oss
```

O `--oss` conversa com o módulo 13 do curso (Claude Code com Ollama): a mesma ideia de rodar o agente com um modelo local.

Automação com *codex exec*

`codex exec` (ou `codex e`) roda uma tarefa **sem interação humana** e termina sozinho. É o modo para scripts, hooks e pipelines de CI.

```
# tarefa simples, com escrita no workspace
codex exec -s workspace-write "Atualize o CHANGELOG com os
commits da semana"

# prompt vindo do stdin
cat tarefa.md | codex exec -

# eventos em JSON (um por linha) para outro programa
codex exec --json "Liste os TODOs do projeto"

# salva só a resposta final num arquivo
codex exec -o resumo.txt "Resuma o README"

# continua a última execução deste diretório
codex exec resume --last "Agora rode os testes"
```

OPÇÃO	USO
<code>--output-schema</code>	Arquivo JSON Schema que a resposta final precisa seguir
<code>--ephemeral</code>	Não grava a sessão em disco
<code>--skip-git-repo-check</code>	Permite rodar fora de um repositório Git
<code>--ignore-user-config</code>	Ignora o config.toml pessoal (bom para CI)

`--full-auto` está **descontinuado**. Use `--sandbox workspace-write`. Se a submissão falhar, o Codex encerra com código diferente de zero, o que facilita tratar erros no CI.

Sessões, nuvem e *MCP*

A CLI se conecta às outras superfícies do Codex e às suas ferramentas.

SESSÕES

codex resume

Reabre um chat recente do repositório atual.
Use `--last` para pular o seletor e `--all` para incluir outros diretórios.

```
# retomar o último chat deste repositório
codex resume --last

# enviar tarefa à nuvem com 2 tentativas
codex cloud exec --env ENV_ID --attempts 2 "Corrija o bug #42"

# autocompletar no zsh
codex completion zsh > "${fpath[1]}/_codex"
```

NUVEM

codex cloud

Lista chats da nuvem, envia tarefas para um environment e aplica o resultado no repositório local (experimental, aula 1).

FERRAMENTAS

codex mcp

Adiciona servidores MCP locais ou remotos, autentica e mostra as ferramentas antes do uso.

SUBAGENTS

Para investigações grandes, peça ao Codex que delegue partes a **agentes especializados** e traga as conclusões para a sessão principal (`/agent` alterna entre as threads).

`codex doctor` gera um diagnóstico da instalação, da configuração, da autenticação e do Git. Rode antes de abrir um chamado de suporte.

config.toml e *AGENTS.md*

A CLI e a extensão do VS Code (aula 3) compartilham as mesmas camadas de configuração. Os padrões pessoais ficam em `~/.codex/config.toml`, e cada projeto pode ter o seu `.codex/config.toml`, que só é carregado em projetos confiáveis.

`~/.CODEX/CONFIG.TOML` (EXEMPLO)

```
model = "gpt-6-sol"
model_reasoning_effort = "medium"
sandbox_mode = "workspace-write"
approval_policy = "on-request"
```

```
# servidor MCP local
[mcp_servers.docs]
command = "npx"
args = ["-y", "@acme/docs-mcp"]
```

PRECEDÊNCIA

1. Flags da CLI e `-c`
2. `.codex/config.toml` do projeto (a pasta mais próxima vence)
3. `~/.codex/config.toml` pessoal, com o perfil `-p` aplicado por cima

AGENTS.MD

As instruções persistentes do projeto: comandos de teste e lint, convenções, regras de review. Gere com `/init`. É o equivalente ao `CLAUDE.md`.

Nomes de modelo mudam com frequência. Confira os disponíveis com `/model` e valide a configuração com `/debug-config`.

Teste *rápido*

1. Qual combinação deixa o Codex editar o projeto, mas pedir permissão para ir além dele?

A) `-s read-only -a never`

B) `-s workspace-write -a on-request`

C) `--yolo`

D) `-s danger-full-access`

B. O `workspace-write` permite editar no workspace, e o `on-request` faz o Codex perguntar quando precisa passar desse limite.

2. Numa pipeline de CI, qual comando roda uma tarefa sem interação e grava só a resposta final num arquivo?

A) `codex resume --last`

B) `codex --search`

C) `codex exec -o saida.txt "..."`

D) `codex cloud list`

C. O `codex exec` é o modo não interativo, e o `-o` (`--output-last-message`) grava a mensagem final num arquivo.

3. No composer, o que faz uma linha que começa com `!` ?

A) Envia o prompt como urgente

B) Roda um comando de shell local

C) Cancela a rodada atual

D) Abre um chat lateral

B. Ela roda o comando no shell local, respeitando o sandbox e as aprovações em vigor.

4. Qual slash command migra configurações e chats do Claude Code para o Codex?

A) `/init`

B) `/resume`

C) `/import`

D) `/app`

C. O `/import` abre o seletor de importação para Claude Code ou Cursor (configuração, projetos e chats recentes).

Uma CLI para o dia a dia *e para automação*

A Codex CLI instala com um comando e faz login pela conta do ChatGPT. Ela combina uma interface de terminal rica (@, !, slash commands) com controle fino de sandbox e aprovações, e com o `codex exec` para scripts e CI. Com Cloud, app desktop, VS Code e CLI, você viu as quatro interfaces do Codex neste módulo.

`install.sh · npm · brew`

`-s · -a`

`/import`

`codex exec`

`config.toml`

Próximo passo → escolha a interface que combina com o seu fluxo e compare o Codex com o Claude Code num mesmo repositório

Fontes: [learn.chatgpt.com/docs](https://learn.chatgpt.com/docs/codex/cli) (codex/cli · developer-commands?surface=cli · slash commands · sandboxing · non-interactive-mode · config-file/config-basic · auth). Comandos, flags e modelos podem mudar; confira a documentação oficial.